



3



4



480 sp

عملي

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

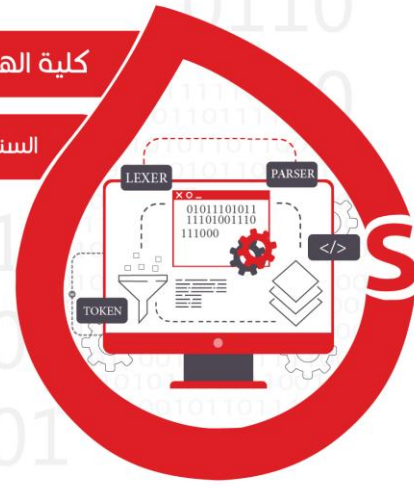
Code Generation

م.وسيم البزرة، آية شحادة

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 29/6/2025

الترجمات



مفهوم توليد الكود (Code Generation)

هي المرحلة الأخيرة في بناء المترجم، والهدف منها تحويل شجرة الـ AST (الشجرة التجريدية للنحو) إلى كود حقيقي بلغة الهدف (JavaScript أو PHP أو غيرها)



الخطوات الرئيسية لتوليد الكود:

1. فهم اللغة المصدر واللغة الهدف:

- اللغة المصدر: هي اللغة التي بنيت فيها الـ AST غالباً
- اللغة الهدف: هي اللغة التي سيتم توليد الكود منها (مثل JavaScript، HTML، أو PHP)
- يجب إجراء عملية مطابقة (Matching) بين مفاهيم اللغة المصدر ومكافئاتها في اللغة الهدف

2. إعداد هيكل الشجرة - AST Root

- يتم بناء جذر الـ AST من خلال Parser.
- كل عقدة (Node) تمثل جزءاً من البرنامج (مثل زر، نص، صورة، حالة state)
- لكل نوع من العناصر، يتم إنشاء كلاس يمثل بنيته المنطقية، ويتضمن هذا الكلاس دالة مسؤولة عن توليد الكود (generate).

3. تحضير ملفات الواجهات (Views)

- غالباً يتم توليد ثلاث واجهات أساسية:
- List View: لعرض قائمة العناصر.
- Details View: لعرض تفاصيل عنصر معين.
- Add View: لإضافة عنصر جديد.
- كل ملف ناتج يمكن أن يكون:
- إما ملف JavaScript + HTML.
- أو ملف PHP يحتوي HTML داخله.

4. إضافة (Navigation) وإدارة الحالة (State Management)

- (navigation) تنقل الصفحات بين بعضها.
- في حال وجود روابط مثل [routerLink] ، يتم توليد تعليمات تربط بين الصفحات أو تنقل المستخدم عبر أجزاء التطبيق.
- (state) تحتفظ بحالة المستخدم أو العناصر .
- مثلاً: عندما يُدخل المستخدم اسمه في حقل إدخال، يجب أن يتم تخزين هذه القيمة وربطها بعنصر منطقي في التطبيق.

5. استخدام FileWriter لتوليد الملفات

- نطبع الأكواد باستخدام FileWriter بنفس الطريقة التي نطبع بها رسائل الأخطاء.
- نكتب الكود المولّد في ملفات منفصلة. (page1.js, page2.js...)

6. تشغيل السيرفر لعرض النتائج

- بعد توليد الملفات، يمكننا تشغيلها على سيرفر (Node.js أو PHP server)
- مثلاً نضعها في مجلد `setup` ونشغل السيرفر لعرض الواجهات.

7. كود توليد الكود داخل main

- نمرر البرنامج إلى الـ `codeGeneration`:
- `program.codeGeneration();`
- كل كلاس في الـ AST يجب أن يحتوي على دالة `generate()` تقوم بتوليد الكود الخاص به، وتستدعي `generate()` من الأبناء إذا كان عنده `children`.

8. حالات خاصة في التوليد

- في JavaScript يكون لدي شكل تعريف متغير `let x = 0;` عند التحويل لـ PHP يصبح بالشكل `$x = 0;`
- session في PHP مكافئة لـ `useState` أو `localStorage` في JS .
- if نستخدمها للتحقق من الشروط وعدم توليد أشياء لا نحتاجها (مثلاً إذا كان الحقل `null`).



- يمكن وضع قاعدة توليد الحالة (state) إما في ملف منفصل (أفضل) أو بنفس الملف.
- التاغات مثل `<TextField>` في DSL تتحول إلى `<input type="text">` في HTML.
- الخطأ (error handling) يمكن توليده بناءً على تحقق من الشجرة (مثلاً إن كان اسم الحقل مكرراً).

ملخص:

- الـ AST = تمثيل منطقي للواجهة أو البرنامج.
- كل كلاس فيه `generate()` → يولد الكود المناسب.
- نربط مع `FileWriter` لطباعة الملفات.
- نضيف `navigation` و `state management` يدوياً أو عبر قواعد.
- نستخدم سيرفر بسيط لتشغيل الملفات بعد توليدها.



أمثلة على تحويل الكود من HTML إلى Angular :

1.

HTML:

```
<div style = "width: 100px; height: 100px; background - color: blue;" ></div >
```

Angular:

```
<div [style.width.px] = "100" [style.height.px] = "100" style = "background - color: blue;" ></div >
```

2.

HTML:

```
<p style = "padding: 10px;" > Hello </p >
```

Angular:

```
<p [ngStyle] = "{ padding: '10px' }" > Hello </p >
```

3.

Angular:

```
<app - root ></app - root >
```

HTML:

```
<body >
```

```
<header > Title </header >
```



```
< main >...</main >  
</body >
```

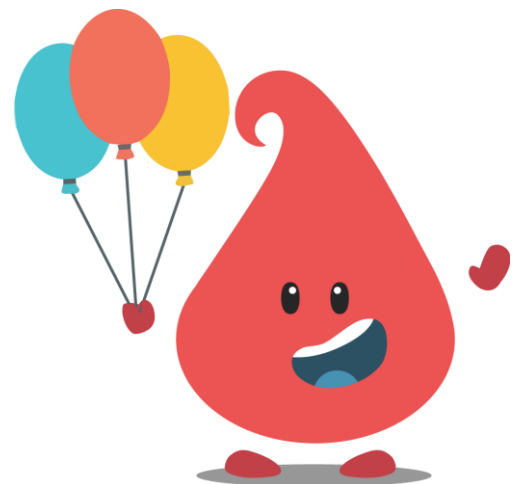
.4

Angular:

```
< ul >  
< li *ngFor = "let item of items" > {{ item }} </li >  
</ul >
```

HTML:

```
< ul id = "list" ></ul >  
< script >  
const items = ["A","B","C"];  
const ul = document.getElementById('list');  
items.forEach(item => {  
  const li = document.createElement('li');  
  li.textContent = item;  
  ul.appendChild(li);  
});  
</script >
```



وهكذا نصل إلى ختام رحلتنا مع مادة المترجمات.... نرجو أنها كانت خفيفة ومفيدة. لا تنسونا من صالح دعائكم